

Live Memory Analysis using Volatility

Dr. Manu V. T.

March 3, 2026

Agenda

- 1 Introduction to Memory Forensics
- 2 Memory Forensics Steps
- 3 Memory Acquisition
- 4 The Volatility Framework
- 5 Methodology
- 6 Process Analysis
- 7 Handles and DLL Analysis
- 8 Dumping Artifacts
- 9 Network and Registry Analysis
- 10 Command History
- 11 Key Takeaways
- 12 Core Volatility Commands
- 13 Case Study: Detecting a Reverse Shell
- 14 Profiles in Volatility 2
- 15 Symbol Tables in Volatility 3
- 16 Structural Comparison
- 17 Deep Technical Role
- 18 Conceptual Summary
- 19 Conclusion

Why Memory Forensics?

- **Volatility:** RAM is transient. Once the power is gone, the data is gone.
- **Stealthy Malware:** Modern threats often reside only in memory (fileless malware) to avoid detection by traditional disk-based antivirus.
- **Rich Data Source:** RAM contains:
 - Running processes and network connections.
 - Loaded DLLs and kernel modules.
 - Encryption keys and passwords in plaintext.
 - Fragments of chat messages and browser history.

Step 1: Memory Acquisition

- **Definition:** This involves acquiring (or dumping) the memory of a target machine to disk.
- **Target Scenarios:**
 - **Infected System:** A machine on your network suspected of being compromised.
 - **Malware Analysis Lab:** A controlled lab environment where a malware sample was executed.
- **Goal:** To preserve the current state of the RAM before the system is powered down or evidence is overwritten.

Memory Acquisition

- Tool: DumpIt
- Produces raw memory image (.raw/.mem)
- Must be acquired early in investigation
- Preserve forensic integrity

Step 2: Memory Analysis

- **Definition:** After you dump the memory to disk, this step involves analyzing the dumped memory to find and extract forensic artifacts.
- **Key Objectives:**
 - Reconstruct the state of the OS at the time of capture.
 - Locate evidence of malicious activity (e.g., hidden processes, injected code).
 - Extract sensitive data like credentials or decryption keys.

What is Volatility?

- The world's most widely used open-source framework for incident response and malware analysis.
- Written in **Python**.
- Supports Windows, Linux, and macOS.
- **Volatility 2 vs. 3:**
 - Volatility 2 relies on "profiles" (Kernel symbols).
 - Volatility 3 (the current standard) uses "symbol tables" and automates much of the OS identification logic.

Click here: [Volatility Framework Homepage](#)

The Forensic Workflow

- ① **Acquisition:** Capture the RAM image from the live system (e.g., using FTK Imager, DumpIt, or LiME).
- ② **Identification:** Determine the OS and kernel version of the image.
- ③ **Analysis:** Run plugins to extract specific artifacts.
- ④ **Reporting:** Correlate findings to reconstruct the timeline of the attack.

Volatility Overview

- Open-source memory forensics framework
- Requires correct OS profile
- Supports numerous forensic plugins

Basic Usage:

```
python vol.py -f memory.img --profile=Win7SP1x86 pslist
```

Enumerating Processes

- pslist – Active process listing
- psscan – Pool tag scanning
- psxview – Cross-view detection

Detecting Hidden Processes

- Direct Kernel Object Manipulation (DKOM)
- Removes process from `ActiveProcessLinks`
- `psxview` compares multiple views

Listing Process Handles

- Plugin: handles
- Identifies:
 - Files
 - Registry keys
 - Mutexes
 - Events

Listing and Detecting DLLs

- dlllist – Lists loaded modules
- ldrmodules – Detects hidden/unlinked DLLs
- Indicators:
 - InLoad
 - InInit
 - InMem flags

Dumping Executables and DLLs

- procdump – Extract process executable
- dlldump – Extract DLLs
- Enables further static analysis

Network Connections

- nmap plugin
- Identifies:
 - Active connections
 - Listening ports
 - Remote IP addresses
 - Associated PIDs

Registry and Services

- printkey – Inspect registry
- svcsan – Enumerate services
- Detect malicious persistence

Extracting Command History

- cmdscan – Extract cmd.exe history
- consoles – Extract command output buffers
- Reveals attacker activity

Key Concepts

- Memory contains critical forensic artifacts
- Cross-view analysis is essential
- Detect DKOM and hidden processes
- Correlate process, DLL, network, registry artifacts

The Evolution of Volatility

- **Volatility 2.6:** The industry standard for over a decade, built on Python 2.7.
- **Volatility 3:** A ground-up rewrite designed for modern systems and automation.
- **Objective:** Understanding the shift from static profiles to dynamic symbol resolution.

Core Architectural Comparison

Feature	Volatility 2.6	Volatility 3
Language	Python 2.7 (Legacy)	Python 3.x (Active)
Codebase	Monolithic	Modular/OO
Dependencies	Heavy (Distorm3, etc)	Minimal Core
OS Support	Static Profiles	Symbol Tables
Performance	Slower	Highly Optimized

Common Commands (Volatility 3)

Process Listing

```
# List all processes
python3 vol.py -f mem.dmp windows.pslist

# List processes including hidden/terminated ones
python3 vol.py -f mem.dmp windows.psscan

# View process tree hierarchy
python3 vol.py -f mem.dmp windows.pstree
```

Network Connections

```
# Show active and closed network connections
python3 vol.py -f mem.dmp windows.netscan
```

Advanced Malware Detection

- **Malfind:** Locates injected code and unbacked memory regions.
- **Yarascan:** Scans memory for specific signatures or patterns.
- **Dlllist:** Shows all DLLs loaded by a specific process.

Example: Finding Injected Code

```
python3 vol.py -f mem.dmp windows.malfind
```

Reverse Shell Scenario

- ① **Initial Check:** Run `netscan` to find suspicious outbound connections (e.g., port 4444).
- ② **Identify Process:** Note the PID associated with the suspicious connection.
- ③ **Inspect Parent:** Use `pstree` to see if `cmd.exe` or `powershell.exe` is a child of a web server or an unusual process.
- ④ **Dump Memory:** Use `procdump` to extract the executable for further static analysis.

The "Profile" Revolution

Volatility 2: Profiles

- Hardcoded offsets.
- Breaks with OS updates (KB patches).
- Requires manual generation for Linux.

Volatility 3: Symbols

- JSON Symbol Tables.
- Fetched automatically via GUID/Banner.
- Future-proof architecture.

Concept of Profiles (Volatility 2)

A **Profile** is a structured description of:

- Target OS version
- Architecture (x86 / x64)
- Kernel data structure layouts
- Field offsets within structures
- Type definitions

It provides the semantic mapping required to interpret raw memory bytes as structured kernel objects.

Why Profiles Exist

Raw memory images are just byte streams.

Example pointer:

```
0xffffffffa8003d4b060
```

To interpret this as an `_EPROCESS` object, Volatility must know:

- Offset of `UniqueProcessId`
- Offset of `ActiveProcessLinks`
- Size of `_LIST_ENTRY`
- Alignment rules
- Pointer size (4 vs 8 bytes)

All of this comes from the profile.

Technical Composition: VTypes

Profiles contain **VTypes** (Volatility Types).

Example (simplified):

```
'_EPROCESS': [ 0x2e0, {  
    'UniqueProcessId': [0x180, ['unsigned long']],  
    'ActiveProcessLinks': [0x188, ['_LIST_ENTRY']],  
}]
```

Each field specifies:

- Offset
- Type
- Nested structure (if applicable)

System Metadata in Profiles

Profiles also include:

- OS name and Service Pack
- Architecture
- DTB offsets
 - DTB (Directory Table Base) is the physical address of the top-level page table for a process.
 - A process does not “contain” memory directly.
Instead:
A process owns an address space, and that address space is defined by its page table hierarchy.
 - It is sometimes referred to as:
 - Page Directory Base
 - Page Directory Pointer (legacy terminology)
 - Each process has its own DTB.
- KDBG signature offsets
 - KDBG (Kernel Debugger Data Block) is a Windows kernel structure used by the debugger.

What DTB Actually Represents

- DTB is a physical pointer to the paging structure used to translate virtual addresses.
- A memory image is raw physical memory.
- To interpret a virtual address like: **0xfffff80002abc000**
- Volatility must:
 - Obtain DTB
 - Walk the page tables
 - Translate VA → PA
 - Read physical memory
- Without DTB, virtual memory reconstruction is impossible.

Address Space Configuration

Profiles define:

- Page table interpretation
- Virtual $\rightarrow$ Physical translation
- Kernel space mapping

How Profiles Are Built

Profiles are generated from:

- Windows PDB files:

For the Windows kernel (`ntoskrnl.exe`), the PDB file contains:

- `_EPROCESS` layout
 - `_ETHREAD`
 - `_DRIVER_OBJECT`
 - `_KTHREAD`
 - `_OBJECT_HEADER`
 - Global variables (e.g., `PsActiveProcessHead`)
 - Function addresses
- Microsoft debugging symbols
 - Extracted kernel type information

Tools used:

```
pdbparse  
dwarf2json
```

Limitations of Volatility 2 Profiles

- Static and rigid
- Must match exact OS build
- Minor patches may shift offsets
- Manual profile generation required

Concept of Symbol Tables (Volatility 3)

A **Symbol Table** describes:

- Types and structures
- Enums
- Global variables
- Kernel symbols
- Function addresses

Unlike profiles:

- Modular
- Dynamically loaded
- JSON-based
- Multi-OS friendly

Intermediate Symbol Format (ISF)

Volatility 3 uses JSON-based ISF.

Example (simplified):

```
{
  "types": {
    "_EPROCESS": {
      "fields": {
        "UniqueProcessId": {
          "offset": 0x2e0,
          "type": "unsigned long"
        }
      }
    }
  }
}
```

Layered Architecture in Volatility 3

Physical Layer

↓

Virtual Layer

↓

Kernel Module Layer

↓

Symbol Table

Symbol tables attach to memory layers to reconstruct objects.

Dynamic Symbol Resolution

Volatility 2:

```
--profile=Win7SP1x64
```

Volatility 3:

- Detects kernel banner
- Matches correct symbol table
- Loads JSON symbols automatically
- Applies types dynamically

Profiles vs Symbol Tables

Feature	Volatility 2	Volatility 3
Format	Python VTypes	JSON (ISF)
Flexibility	Static	Dynamic
OS Matching	Manual	Automatic
Multi-OS	Limited	Strong
Maintenance	Difficult	Modular
Kernel Symbols	Limited	Full Support

Object Reconstruction

Profiles / Symbol Tables allow reconstruction of:

- _EPROCESS
- _ETHREAD
- _DRIVER_OBJECT
- _FILE_OBJECT

Linked List Traversal

Example:

`ActiveProcessLinks → _LIST_ENTRY`

Without symbol metadata:

- Cannot determine pointer offsets
- Cannot safely traverse kernel lists

Rootkit Detection Dependency

Cross-view plugins rely on:

- Accurate structure offsets
- Correct pool tags
- Valid object headers

Incorrect symbols cause:

- False positives
- Missed detections

Kernel Dependency

Profiles / Symbol Tables are kernel-version dependent:

- Structure layouts change
- Padding changes
- Field ordering changes
- Security mitigations alter structures

Even minor patches may shift offsets.

Profile (Volatility 2)

Static blueprint of kernel memory layout.

Symbol Table (Volatility 3)

Dynamic type system enabling structured object interpretation.

If symbols are incorrect:

- pslist fails
- netscan returns garbage
- dlllist misinterprets structures
- DKOM detection becomes unreliable

Workflow & Syntax Changes

Volatility 2 (Manual):

Legacy Syntax

```
python vol.py -f mem.raw --profile=Win7SP1x64 pslist
```

Volatility 3 (Automated):

Modern Syntax

```
python vol.py -f mem.raw windows.pslist
```

Note the use of Namespacing (windows.plugin).

Major Technical Advancements

- **Automated Layering:** Detects file formats (VMware, Crash Dumps, Raw) automatically.
- **Plugin Namespacing:** Prevents overlap between OS-specific commands.
- **Library Independence:** Can be integrated directly into larger Python forensic pipelines.
- **No Kdbgscan:** Kernel headers are found during initial file loading.

Persistence of Volatility 2.6

Despite being legacy, version 2 remains useful for:

- **Niche Malware:** Specific plugins (Stuxnet/Zeus parsers) not yet ported.
- **Legacy OS:** Extremely high reliability for Windows XP/2003 forensics.
- **Air-gapped Labs:** Local profiles avoid the need for external Symbol Servers.

Volatility 3 is the recommended standard for modern Incident Response.

Volatility 2 remains a critical historical tool for legacy systems.

Conclusion and Best Practices

- **Do Not Reboot:** Rebooting a compromised machine destroys the most valuable evidence.
- **Order of Volatility:** RAM should be collected **first** in the digital evidence collection process.
- **Continuous Learning:** The Volatility community frequently releases new plugins for emerging threats.